

# 实验 2 - 全连接神经网络

数据科学与大数据技术专业

第 4 周

## 1 实验目标

本实验旨在使学生掌握以下内容：

- 手动构建  $L$  层神经网络, 理解其结构与实现细节.
- 利用非线性激活函数 (如 ReLU) 提升模型性能;
- 实现易于使用的神经网络类, 用于图像分类任务 (例如猫/非猫分类);

## 2 实验环境

- 操作系统: Mac OS、Windows、Linux
- 工具: Visual Studio Code、Jupyter Notebook、Anaconda
- 编程语言: Python

## 3 实验步骤

### 3.1 步骤 1: 理论学习与资料阅读

- 熟悉 2 层神经网络与深层神经网络之间的区别;
- 理解各层数据表示的符号意义:  
 $a^{[l]}$ : 第  $l$  层的激活值;  
 $W^{[l]}$ 、 $b^{[l]}$ : 第  $l$  层参数;  
 $x^{(i)}$ : 第  $i$  个训练样本.
- 掌握前向传递公式:  
 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ ,  
 $a^{[l]} = g^{[l]}(z^{[l]})$ , 其中  $g^{[l]}$  是激活函数.

### 3.2 步骤 2: 实现深层神经网络核心函数

- 安装缺失的依赖: 确保实验环境中安装了所需的 Python 库, 包括 numpy、matplotlib、h5py 等.
  - 使用以下命令安装依赖:

```
pip install numpy matplotlib h5py
```

– 如果使用 Anaconda, 可以运行:

```
conda install numpy matplotlib h5py
```

- 实现前向传播: 构建多层网络的激活值传播, 其中使用 ReLU 作为隐藏层激活函数;

ReLU (Rectified Linear Unit, 修正线性单元) 是一种常用的激活函数, 广泛应用于深度学习模型中. 它的数学表达式为  $f(x) = \max(0, x)$ , 即当输入值为正时, 输出为输入值本身; 当输入值为负时, 输出为 0. ReLU 的优点包括:

- 计算简单高效, 适合大规模神经网络的训练.
- 能够缓解梯度消失问题, 从而加速模型的收敛.
- 在隐藏层中引入非线性, 使得神经网络能够学习复杂的特征表示.

然而, ReLU 也存在一些缺点, 例如可能导致“神经元死亡”问题 (即某些神经元的输出恒为 0) .

- 实现反向传播: 计算每一层的梯度, 为参数更新做准备;  
反向传播公式如下:

$$\begin{aligned}\delta^{[L]} &= \nabla_a \mathcal{L} \odot g'^{[L]}(z^{[L]}), \\ \delta^{[l]} &= (W^{[l+1]})^T \delta^{[l+1]} \odot g'^{[l]}(z^{[l]}), \\ \frac{\partial \mathcal{L}}{\partial W^{[l]}} &= \delta^{[l]} (a^{[l-1]})^T, \\ \frac{\partial \mathcal{L}}{\partial b^{[l]}} &= \delta^{[l]}.\end{aligned}$$

其中, 符号  $\odot$  表示逐元素乘法 (也称为 Hadamard 乘积) . 对于两个形状相同的矩阵或向量  $A$  和  $B$ , 逐元素乘法的结果是一个与  $A$  和  $B$  形状相同的矩阵或向量, 其每个元素由对应位置的元素相乘得到, 即:

$$(A \odot B)_{ij} = A_{ij} \cdot B_{ij}$$

这种操作在神经网络的反向传播中非常重要, 用于将梯度与激活函数的导数逐元素结合, 从而计算每一层的误差项.

- 在输出层可根据分类任务选择合适的激活函数 (例如 softmax) .

### 3.3 步骤 3: 构建神经网络类

- 基于步骤 2 中的函数, 构建一个整合各环节的神经网络类;
- 该类需支持模型初始化、前向传播、反向传播及参数更新;
- 实现训练和预测函数, 便于调用训练数据和测试数据进行模型评估.

### 3.4 步骤 4：应用实例——猫/非猫图像分类

- 利用所构建的深层神经网络类, 完成猫/非猫图像分类任务;
- 比较与之前 Logistic 回归模型的分类效果, 观察准确率上的改进;
- 分析多层网络在特征提取和分类方面的优势.

## 4 实验作业

完成本实验后, 请提交以下材料:

1. 补全 Jupyter Notebook 中的代码, 确保能够正确运行并完成实验目标;
2. 尝试不同的激活函数 (如 Sigmoid、Tanh 等), 分析其对分类效果的影响;
3. 使用 PyTorch 框架实现猫/非猫图像分类任务 (其他分类任务也可), 并提交代码.